

Deepfake Detection: A Convolutional Neural Network Approach with Strategic Data Handling

Tanzeem Tahmeed Reza

Electronics and Telecommunication
Engineering
Chittagong University of Engineering
and Technology
Chattogram, Bangladesh
tanzeemtahmeed01@gmail.com

Muhammad Junayed

Electronics and Telecommunication
Engineering
Chittagong University of Engineering
and Technology
Chattogram, Bangladesh
u2008023@student.cuet.ac.bd

S.M. Shahriar

Electronics and Telecommunication
Engineering
Chittagong University of Engineering
and Technology
Chattogram, Bangladesh
sayeem26s@gmail.com

Mohammed Tawshif Hossain

Electrical and Electronics Engineering
Chittagong University of Engineering
and Technology
Chattogram, Bangladesh
tawsifturjoejee@gmail.com

Annisha Wazed

Commercial Aviation
L3Harris Technology
Tampa, Florida
annishawazed19@gmail.com

Nursadul Mamun

Electronics and Telecommunication
Engineering
Chittagong University of Engineering
and Technology
Chattogram, Bangladesh
nursad.mamun@cuet.ac.bd

Abstract—Deepfakes represent a significant challenge in the digital era, leveraging advanced generative models such as Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs), and Diffusion Models to create hyper-realistic fake media. While these technologies have legitimate applications, they also pose significant risks, including misinformation, privacy breaches, and identity theft. This study proposes a Convolutional Neural Network (CNN)-based framework for deepfake detection, tailored to the IEEE Signal Processing Cup dataset. A strategic data-splitting approach was employed, dividing the fake images into four subsets while keeping the real images consistent to ensure efficient training. The CNN architecture features progressive convolutional blocks with dropout layers and batch normalization to improve generalization and mitigate overfitting. The model achieved high validation accuracy across all subsets, with the best-performing phase yielding 97.59% accuracy, demonstrating robustness in detecting forgery patterns. Metrics such as precision, recall, F1-score, and Equal Error Rate (EER) validate the approach. This work underscores the scalability, simplicity, and effectiveness of the proposed model in addressing the challenges of deepfake detection. By leveraging optimized architectures and strategic approaches, it demonstrates efficient handling of large-scale datasets, making a significant contribution to safeguarding digital integrity.

Keywords—deepfake, convolutional neural networks

I. INTRODUCTION

In recent years, the rapid advancements in artificial intelligence and deep learning have facilitated the creation of highly realistic fake media, commonly known as Deepfakes. These technologies can manipulate videos by merging, replacing, or superimposing images and video clips, producing hyper-realistic content that often includes altered visual and audio elements [1]. Fake videos can be weaponized to spread misinformation, tarnish reputations, or create explicit content without consent, causing widespread harm [2, 3]. Furthermore, the ease of access to tools and libraries for generating Deepfakes has lowered the barrier for malicious actors to produce and disseminate fake media [1,4].

Deepfake detection has become a critical area of research due to the inability of humans to discern synthetic media from authentic content without assistance [5]. Researchers have proposed numerous approaches for detecting manipulated media, ranging from spatial and frequency analysis of images to advanced multimodal techniques that combine temporal and audio cues [6]. Despite these advances, challenges persist due to excessive compression, down sampling, and the application of multiple editing operations, all of which obscure detectable traces of manipulation [7].

This report focuses on developing a convolutional neural network (CNN)-based framework to effectively detect Deepfake content, leveraging the IEEE Signal Cup dataset. Our approach focuses on the unique patterns and flaws created during Deepfake generation, aiming to handle challenges like image compression and small, hard-to-detect edits.

Our contribution includes:

- A CNN framework optimized for high accuracy in detecting real or fake images and also promising for generalized unseen data.
- A comprehensive evaluation and benchmarking on the IEEE Signal Processing Cup Dataset.
- A strategy to handle huge chunk of datasets with simplistic adopted CNN model.

II. METHODOLOGY

This section is about the data processing and model implementation. Our aim was to achieve a robust and simpler CNN architecture that could handle a huge chunk of images with ease. Initially the whole dataset has been trained with the proposed CNN model but the execution time was too lengthy that we did not move in that way further but came up with the idea of splitting the dataset within four individual folders containing ¼th of the fake images in each and trained the subfolders with the same CNN architecture.

The following subsections explain the three main parts of our method: 1) Data processing before feeding into the CNN structure; 2) Proposed approach for image classification; 3) the proposed CNN architecture.

A. Dataset

For this research, a collection of publicly available datasets that have been created for the DeepfakeBench evaluation, has been provided. The standard datasets are preprocessed and came in two individual folders, named train and valid with real and fake subfolders. The folder for training got 42690 real images in the 'train_real' subfolder and the subfolder 'train_fake' contained 219470 fake images. Also, we have got access to validation files, where there were 1548 real images and 1524 fake images for performing the validation scores.



Fig. 1 : Real and Fake Images from the dataset

B. Data Processing

a) Dataset Splitting:

The images in number were large enough and it was quite harder to train the whole dataset in a single trial with limited resources. That is why we came up with the strategy of splitting up the fake images from training folder into 4 phases. Each of the phase contained about one-fourth of the 219470 fake images provided and each of them had the 42690 real images with the divided folders. Our data splitting has been shown in table 1.

TABLE I. TRAINING DATA SPLITTING

Folder Name	Data Size
Train_1 • Real_1 • Fake_1	Real_1: 42690 Fake_1: 54500
Train_2 • Real_2 • Fake_2	Real_2: 42690 Fake_2: 54500
Train_3 • Real_3 • Fake_3	Real_3: 42690 Fake_3: 54500
Train_4 • Real_4 • Fake_4	Real_4: 42690 Fake_4: 55970

The reason behind the data splitting is to make the model run time lesser than that of the time would take to run with the whole dataset. Each of the phase has been trained with the same CNN architecture and parameters to keep the similarity and have the almost same performance.

b) Data Augmentation:

To enhance the diversity of the dataset and mitigate overfitting, an image augmentation pipeline was implemented. This process aimed to synthetically expand the dataset by applying a series of random transformations to the real images. Each image was resized to a fixed resolution of 128X128 pixels for uniformity using TensorFlow's preprocessing utilities. Subsequently, the resized images were converted to arrays and subjected to the following augmentation techniques: random horizontal flipping, vertical flipping, brightness adjustment, and contrast variation. These transformations were chosen to simulate real-world variations in image capture conditions, such as changes in illumination and viewing angles. The parameters for augmentation are shown below in table 2.

TABLE II. AUGMENTATION PARAMETERS

Augmentation Type	Parameters
Horizontal flip	Random
Vertical flip	Random
Brightness adjustment	± 0.1
Contrast adjustment	[0.8, 1.2]

The augmentation process was iteratively performed until the desired number of augmented samples was achieved. A robust error-handling mechanism was integrated to ensure continuity of the augmentation process by bypassing images that could not be processed due to unexpected errors. The augmented images were stored in memory for subsequent use in model training.

c) Data generator initialization:

We have introduced a data generator for training models with normalized image inputs where the image resolution was set to 128X128 and the batch size was determined to be 64. The pixel values were normalized from [0,255] to [0,1].

C. Maintaining the Integrity of the Specifications

Our approach is comparably simple in terms of the implementation of deep learning models. The way we followed is that we did stick on a Convolutional Neural Network model as we know that CNNs are out there to help with image classification tasks.

In our strategy, we firstly split the whole fake image dataset given for training into four individual subfolders with real ones in each as shown in table 1. Then the real images have been augmented a little to maintain balance in each subfolder. Furthermore, the CNN model has been designed and complied. After that the separated folders containing the real and fake images are fed as a data-frame through the CNN model which is executed four times for four of the folders each. In this manner the output performance metrics along with submission score file have been gained. The overall methodology is shown in fig.2.

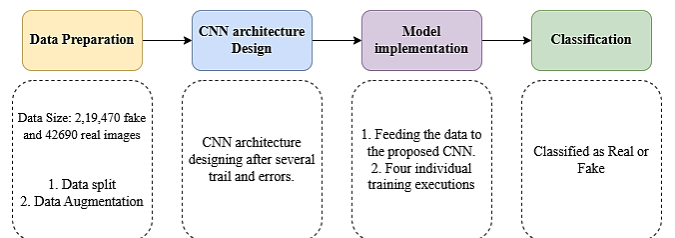


Fig. 2 : Overview of Proposed Methodology

D. Proposed CNN Architecture

The automatic feature extraction process is the basis of Deep Learning. However, the architecture of deep CNN affects its effectiveness. Consequently, we have added a more sophisticated architecture that goes beyond its forebears in terms of execution. However, increased profundity by itself does not ensure better outcomes. Taking care of data imbalance concerns, improving CNN instruction and improving model robustness and generalization are equally essential for achieving peak performance [8]. A little augmentation was essential to meet these requirements. The CNN architecture that we have deployed after several trials and errors is shown below in Fig. 3.

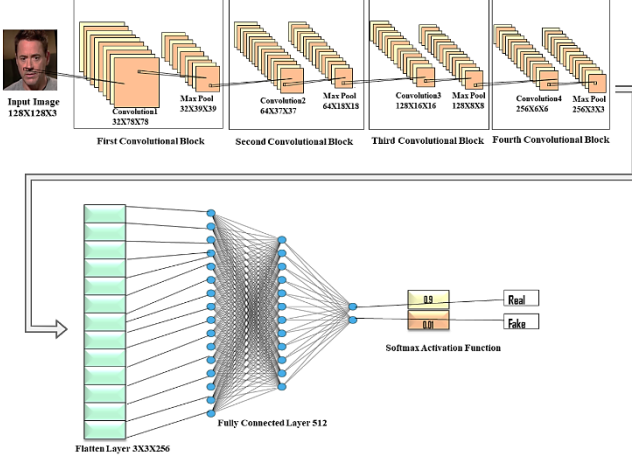


Fig. 3 : Overview of Proposed CNN Architecture

a) Model Overview:

The suggested Convolutional Neural Network (CNN) model was created to effectively categorize images, achieve good generalization, and handle overfitting. In order to provide a small yet meaningful resolution for feature extraction, the input photos were scaled to $128 \times 128 \times 3$. The network comprises four convolutional blocks, each containing a convolutional layer followed by batch normalization, max pooling, and dropout layers. These blocks progressively increased the filter sizes from 32 to 256, enabling the model to capture fine-grained features in initial layers and more abstract patterns in deeper layers.

ReLU activation was employed in all convolutional and dense layers to introduce non-linearity, while a SoftMax activation function in the output layer facilitated classification into two categories. Dropout rates of 30%, 40% and 50% were strategically applied in different layers to mitigate overfittings.

The model was compiled with the Adam optimizer, leveraging a learning rate of 0.001 for adaptive gradient updates. Categorical cross-entropy was chosen as the loss function due to its effectiveness in multi-class classification. Training was performed over 28 epochs with a batch size of 64, balancing computational efficiency and convergence. Performance was evaluated using accuracy as the primary metric and loss as a secondary measure, ensuring robust assessment of the model's effectiveness.

b) Model Summary:

The model summary of our proposed CNN architecture has been given in table 3 for better understanding.

TABLE III. MODEL SUMMARY

Layers	Output	Parameters
Input layer	(80,80,3)	0
Conv2D	(78,78,32)	896
BatchNormalization	(78,78,32)	128
MaxPooling2D	(39,39,32)	0
Conv2D	(37,37,64)	18496
BatchNormalization	(37,37,64)	256
MaxPooling2D	(18,18,64)	0
Dropout	(18,18,64)	0
Conv2D	(16,16,128)	73856
BatchNormalization	(16,16,128)	512
MaxPooling2D	(8,8,128)	0
Dropout	(8,8,128)	0
Conv2D	(6,6,256)	295168
BatchNormalization	(6,6,256)	1024
MaxPooling2D	(3,3,256)	0
Dropout	(3,3,256)	0
Flatten	2304	0
Dense	512	1179136
BatchNormalization	512	2048
Dropout	512	0
Dense (output)	2	1026

So, from the CNN model we got a total parameters of 1,577,514, where the trainable parameters were 1,573,546 and non-trainable parameters were 3968.

c) Model parameters:

The CNN model was designed with an input shape of $128 \times 128 \times 3$, employing the Adam optimizer with a learning rate of 0.001 to ensure adaptive learning. ReLU activation was used in hidden layers for non-linearity, while SoftMax in the output layer facilitated classification. The model was trained using categorical cross entropy loss, a batch size of 64, and 28 epochs, with dropout layers (30%-50%) to reduce overfitting. Performance was evaluated using accuracy and loss metrics.

In order to improve the CNN architecture and its performance for deepfake classification, certain parameters have been established following several trials. The individual parameters that have been used are displayed in Table 4 below.

TABLE IV. MODEL PARAMETERS

Parameters	Value
Input Shape	128 X 128 X 3
Optimizer	Adam
Learning Rate	0.001
Loss Function	Categorical cross entropy
Activation Functions	-ReLU (hidden layer) -SoftMax (output layer)
Batch Size	64
Epochs	28
Dropout	0.3,0.4,0.5

III. RESULTS AND ANALYSIS

A. Experimental Setup

The TensorFlow from Python was chosen as the programming framework, and the models were trained on Kaggle's T4x2 GPU accelerator as there was scarcity of processing units while experimenting. Additionally, Scikit-learn was employed as a data analysis tool to evaluate the performance of the models. The models were trained and tested with different batch sizes (64, 128, 256) and a learning rate of 0.001. Among the batch sizes, with 64 we achieved better outcomes.

B. Performance Evaluation

In this study, accuracy, F1 score, precision, recall and Equal Error Rate were considered as performance measurements. The whole experiment as mentioned earlier has been trained four time with same CNN architecture and we achieved the performance metrics very close to each other for each phase. In this part we will be presenting the overall performance for 28 epochs and 64 batch sizes, as these parameters after several trials have shown promising result.

a) Performance metrics

The overall performance evaluation on validation data has been depicted in the table 5 below.

TABLE V. OVERALL PERFORMANCE

Training phase	Performance Metrics				
	Accuracy	Precision	Recall	F1-score	ERR
Training phase 1	0.9423	0.9122	0.9122	0.9122	0.0872
Training phase 2	0.9571	0.9378	0.9316	0.9318	0.0405
Training phase 3	0.9667	0.9699	0.9699	0.9699	0.0307
Training phase 4	0.9724	0.9759	0.9759	0.9759	0.0234

From the table it is clearly seen that, the fourth split that we have made, has performed better than the other three training phase achieving 97.59% validation accuracy in fake or real classification. The other training phases might have gone through struggling for finding the patterns in the images.

b) Training and Validation Curves

For the four individual execution we got four accuracy vs epochs and four loss vs epochs graphs. Below the curves are described in detailed manner.

For the first split and training the data fig. 5 is shown. In the loss plot, the blue curve refers the training loss, which decreases gradually over the epochs, indicating that the model is learning and fitting the training data. Over 28 epochs, the training loss curve comes nearly to zero and refers a good training loss score. The orange curve represents the validation loss. It shows a downward trend with fluctuations and sharp peaks, particularly at 10-17 epochs, reflecting challenges in generalizing unseen or validated images.

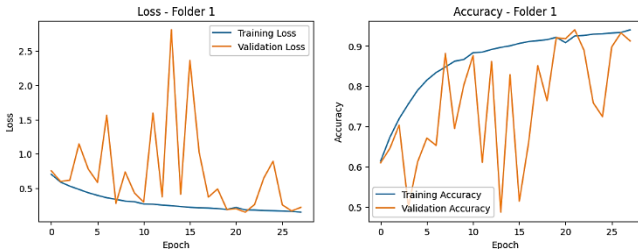


Fig. 5 : Loss curve and accuracy curve over 1st Training

In the accuracy plot, the blue curve refers the training accuracy, which increases and plateaus near to 1 reflecting good performance on the training set. But in the case of validation accuracy, it improves over epochs but show variability and peak values. The peak or variability occurs

due to model generalization learning on the unseen/validated images. But gradually over the epochs, the validation accuracy increases and becomes 0.9122 which refers a good validation score.

The True Positive Rate (TPR) against False Positive Rate (FPR) trade-off across various classification thresholds is portrayed by the ROC curve. The model's capacity to differentiate between classes is assessed by the Area Under the Curve (AUC) shown in fig.6, where 0.97 signifies exceptional performance.

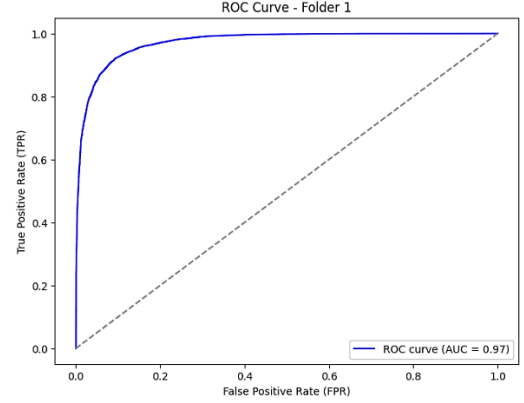


Fig. 6 : ROC Curve over 1st Training

In the second split and model training, fig. 7 depicts the loss and accuracy trends. In the loss curve, it exhibits fluctuations and sharp peaks nearly in 10 to 12 epochs. Despite of some peaks and fluctuations during epoch 15 to 18, the model was able to learn the hidden pattern and the validation loss becomes 0.1756 where training loss is 0.1103.

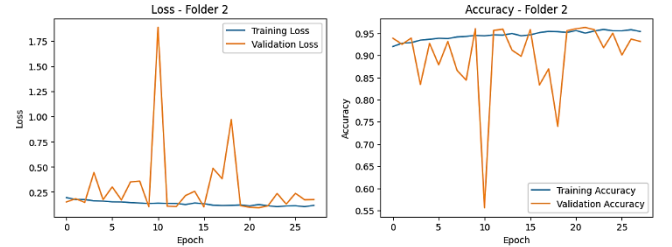


Fig. 7 : Loss curve and accuracy curve over 2nd Training

In the accuracy plot, the blue curve refers the training accuracy, which increases from 0.9423 to 0.9571 reflecting good performance on the training set. But in the case of validation accuracy, it improves over epochs but show sudden variability and peak values. The peak or variability occurs due to model generalization learning on the unseen/validated images. Here over the 28 epochs the model, figured out the pattern gradually and the validation accuracy increases to 0.9316 which refers a good validation score.

The model's ability to distinguish between classes is evaluated using the Area Under the Curve (AUC), as depicted in Fig. 8, with a value of 0.99 indicating outstanding performance.

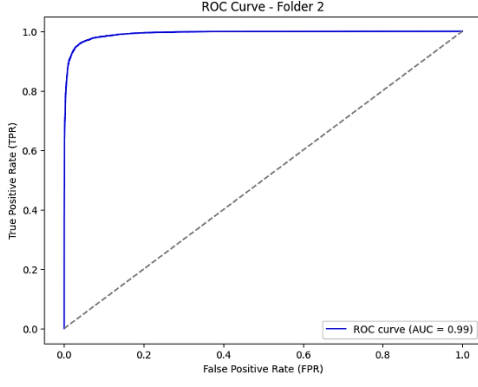


Fig. 8 : ROC Curve over 2nd Training

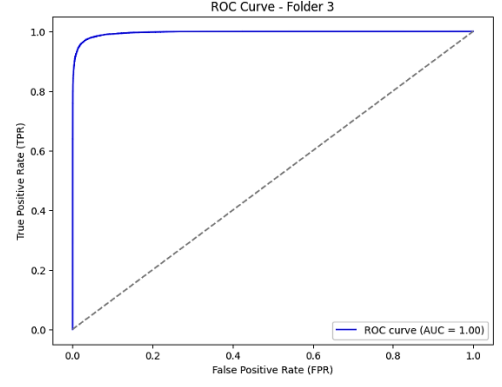


Fig. 11 : ROC Curve over 3rd Training

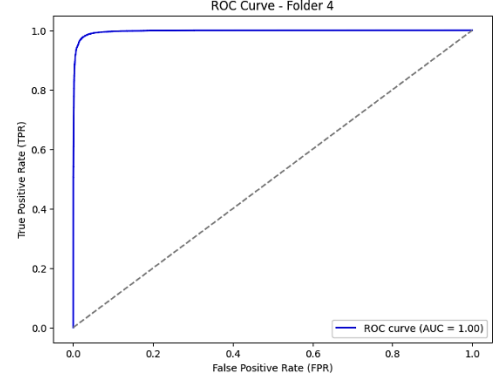


Fig. 12 : ROC Curve over 4th Training

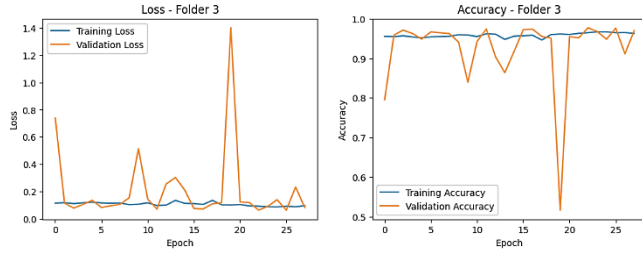


Fig. 9 : Loss curve and accuracy curve over 3rd Training

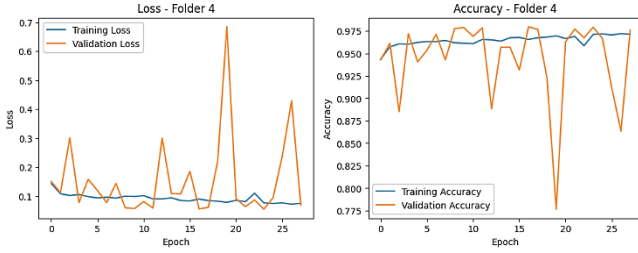


Fig. 10 : Loss curve and accuracy curve over 4th Training

The Area Under the Curve (AUC), as illustrated in fig. 11 and fig.12, with a value of 1 representing exceptional performance for the third and fourth training phase. It indicates the model is has both the sensitivity and specificity of 100%.

From the above, we see that our model over the epochs learns the pattern for each of the training phases, though in some epoch it struggles and finally shows a good result which fulfills the reliability, robustness and in generality.

C. Testing on Validation Data

Testing on the validation data has been done, where the ‘real’ images were mapped to 1 and the ‘fake’ images were mapped to 0. To process both the real and fake folder it took 553ms.

TABLE VI. TESTING PERFORMANCE

Test	Performance Metrics				
	Accuracy	Precision	Recall	F1-score	ERR
Testing phase	0.8551	0.8654	0.8551	0.8542	0.1424

For predicting images, we achieved 0.8551 test accuracy and test loss of 0.4764. Additionally, the Equal Error Rate (EER) after validation, where the FPR equals the False Negative Rate (FNR), is 0.2100. This implies that at this threshold, the model has an equal chance of making false positive and false negative errors, occurring in 21% of cases. The results for testing upon the validation files are given in table 6.

With an Area Under the Curve (AUC) of 0.93, the ROC curve (fig.13) upon the testing of validation files illustrates an outstanding model performance and a great capacity for discrimination between the positive and negative classes. The model performs better the closer the curve is to the upper-left corner.

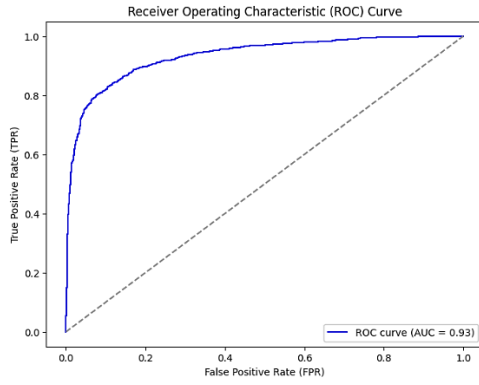


Fig. 13 : ROC Curve over testing

D. Submission Score Generation

The task to submit the scores for the validation data have been produced keeping the main theme in our minds that, the score here indicates how likely the file is to contain ‘real’ images of ‘fake’ images. Here in our approach, the score close to 1 presents the image file to be real and the score close to 0 presents the image file to be fake.

Each of the image file took around 16-20 ms for testing. As there are 3072 files for testing each of the files were tested within the mentioned time.

IV. CONCLUSION

Deepfake detection has been extensively studied, with many state-of-the-art (SOTA) models achieving remarkable performance on specific datasets. However, these models often struggle to generalize to completely new datasets, leading to a significant drop in accuracy. Existing SOTA models or ensemble approaches tend to be dataset-specific, excelling on the images they are trained on but failing to adapt to diverse data or varying types of deepfakes. The present difficulties in deepfake detection may be distilled into three crucial elements: robustness, interpretability, and transferability, even though recent developments have shown encouraging outcomes on cross-dataset testing.

Most existing models are based on convolutional neural networks (CNNs) with modifications to enhance accuracy, yet they often fail when evaluated on different datasets. In this work, we adopt CNN as our base model, focusing on

maintaining its generality and simplicity to ensure robust performance across all types of datasets. To address data imbalance, we split the dataset into four parts and applied data augmentation on real images, balancing the distribution of real and fake images. The CNN model was then trained in four successive stages.

The suggested model showed excellent performance in identifying spatial discrepancies, attaining high precision and recall by utilizing a variety of datasets and sophisticated training techniques. Even with its effectiveness, problems like overfitting and real-time detection still exist. Future research will concentrate on improving generality, adding methods such as adversarial training and ensemble learning, and extending detection capabilities to different kinds of deepfakes. This study illustrates how crucial reliable detection methods are to maintaining the authenticity of digital information and preventing the misuse of deepfakes.

REFERENCES

- [1] Y. Huang, F. Juefei-Xu, Q. Guo, Y. Liu and G. Pu, "FakeLocator: Robust Localization of GAN-Based Face Manipulations," in *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 2657-2672, 2022, doi: 10.1109/TIFS.2022.3141262.
- [2] T. Reiss, B. Cavia, and Y. Hoshen, "Detecting deepfakes without seeing any," *arXiv (Cornell University)*, Jan. 2023, doi: 10.48550/arxiv.2311.01458.
- [3] G. Pei *et al.*, "Deepfake Generation and Detection: A Benchmark and survey," *arXiv (Cornell University)*, Mar. 2024, doi: 10.48550/arxiv.2403.17881.
- [4] Z. Yan, Y. Luo, S. Lyu, Q. Liu, and B. Wu, "Transcending Forgery Specificity with Latent Space Augmentation for Generalizable Deepfake Detection," *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 33, pp. 8984–8994, Jun. 2024, doi: 10.1109/cvpr52733.2024.00858.
- [5] G. Gupta, K. Raja, M. Gupta, T. Jan, S. T. Whiteside, and M. Prasad, "A comprehensive review of DeepFake detection using advanced machine learning and fusion methods," *Electronics (Basel)*, vol. 13, no. 1, p. 95, 2023.
- [6] N. Bonettini, E. D. Cannas, S. Mandelli, L. Bondi, P. Bestagini, and S. Tubaro, "Video Face manipulation detection through ensemble of CNNs," *2022 26th International Conference on Pattern Recognition (ICPR)*, Jan. 2021, doi: 10.1109/icpr48806.2021.9412711.
- [7] H. Lee *et al.*, "The Tug-of-War between deepfake generation and detection," *arXiv (Cornell University)*, Jul. 2024, doi: 10.48550/arxiv.2407.06174.
- [8] A. Hernández-García and P. König, "Further advantages of data augmentation on convolutional neural networks," in *Artificial Neural Networks and Machine Learning – ICANN 2018*, Cham: Springer International Publishing, 2018, pp. 95–103.